

Phase 6 — Engine & Transport

LineTwin · Accenture Innovation Challenge 2026 · Round 2 · Problem Track 4 "DigitalTwin.ai"

Purpose

Build the real-time-paced asyncio tick loop, the single-slot SSE conflation bus, and the REST control surface — the layer that turns a batch simulation into a live, perturbable twin a browser or a `curl` can watch in real time.

This phase found one critical bug via manual verification: a specific FastAPI usage pattern that deadlocked the entire server at 100% CPU under load. It is documented in full below because it was reproduced deliberately, root-caused precisely, fixed, and turned into a regression test — not just patched and moved past.

What was built

Artefact	What it does
<code>src/twin/sim/engine.py</code>	Engine : owns one <code>simpy.Environment</code> + <code>Line</code> , advanced by <code>env.run(until=tick*SIM_DT)</code> from one asyncio task; absolute-deadline anchored sleep with re-anchoring past 5 ticks of lag; <code>ConflationBus</code> (single-slot, <code>asyncio.Condition</code> -based); live control-queue draining; restart via a flag checked only at the top of each tick (never a direct mid-tick mutation); fault frames on simulation exception
<code>src/twin/sim/line.py</code> (extended)	Added <code>_live_multiplier</code> per station and <code>set_cycle_time_multiplier()</code> , so a control command can perturb a station's cycle time mid-run — this did not exist before Phase 6, since the bottleneck multiplier was previously baked in as a fixed constructor-time closure value
<code>src/twin/api/sse.py</code>	The SSE route factory, built exactly to ADR-002's two verified constraints: <code>response_class=EventSourceResponse</code> on the route itself (not a wrapped return value), and <code>raw_data=</code> for pre-serialized JSON (never <code>data=</code>)
<code>src/twin/api/routes.py</code>	<code>create_app()</code> factory; <code>/api/twin/{state,control,heartbeat,restart}</code> , <code>/healthz</code>
<code>tests/test_engine.py</code>	11 tests: tick-timing invariants, control application, restart (including recovering a <i>faulted</i> engine), the conflation bus's pub/sub semantics, and the honest-MISSING-not-fabricated dark-station check
<code>tests/test_api.py</code>	15 tests: REST routes in-process via httpx's ASGI transport; SSE streaming via a real subprocess server (see below for why); the regression test for the bug this

Artefact	What it does
	phase found

The bug: `/api/twin/state` **deadlocked the entire server at 100% CPU**

Manual verification (per this project's standing practice of not calling a route "done" until it has been driven by hand) found this immediately: three concurrent SSE clients plus a burst of `/api/twin/ state` calls left the server unresponsive to **every** request — `curl` timing out completely — with the process pinned at 100% CPU.

Isolating it precisely, not just patching around it: a minimal reproduction outside this project's own routing code confirmed the cause exactly. `/api/twin/state` was declared `async def state() -> Snapshot`, using FastAPI's implicit `response_model` machinery. Measured directly:

Approach	Time for one 30-station snapshot
<code>return engine.bus.latest with -> Snapshot</code> (implicit <code>response_model</code>)	1.35 seconds
<code>Response(content=snapshot.model_dump_json(), ...)</code> (manual)	~2 milliseconds

FastAPI's `response_model` path does not trust an already-valid pydantic instance — it re-validates the whole nested structure (30 stations × `TaggedValue` × `RiskDriver` lists × an enum-keyed `dict`) through its own `jsonable_encoder`, which is dramatically slower than pydantic's own `model_dump_json()`. Because the engine's tick loop and every HTTP request share **one** asyncio event loop, that single 1.35-second call blocked tick production for **over ten ticks' worth of time**. Under concurrent load, each slow response added more lag with nothing left to catch up with — a genuine death spiral, not a one-off slow request.

Fixed: `response_model=None` plus a hand-built `Response` with `model_dump_json()` — exactly the pattern the SSE route already used correctly. Re-verified against a live server:

- A single `/api/twin/state` call: **1.9 ms** (down from 1.35 s).
- 100 rapid sequential calls: **all 200, max 10.3 ms, mean 8.0 ms**, CPU never left single digits.
- The full combined scenario that originally triggered the deadlock (state calls + 3 concurrent SSE clients + a control command, interleaved) — verified clean, CPU staying under 4%.

Turned into a permanent regression test, not just a manual finding: `test_state_endpoint_responds_within_a_tight_latency_bound` (`tests/test_api.py`) asserts a 200 ms bound — six times tighter than a single tick's 125 ms budget, so a future regression (someone reverting to `-> Snapshot` for convenience) fails loudly in CI within milliseconds, instead of being rediscovered by hand against a live server again.

One false alarm during this investigation, reported for completeness: a follow-up stress test briefly showed two requests each stalling for a full 2-second client timeout, immediately after the fix had already been verified working. Retested with a generous 15-second timeout and then a 100-call sequential stress test: all calls completed in single-digit milliseconds with no slow outliers. The original two-call stall did not reproduce and is recorded as most likely local shell/process scheduling noise from interleaved diagnostic commands, not a server defect — noted rather than silently dropped, since a claim of "fixed" should say what was and wasn't re-confirmed.

A second, unrelated finding: in-process ASGI transport cannot test this SSE route

Attempting to test the SSE stream via httpx's in-process `ASGITransport` (the natural first choice, and what every other route in this phase is tested with) hung indefinitely — even breaking out of the client-side reading loop early did not unblock it. This is a test-harness limitation, not a server bug: `ASGITransport` appears to buffer toward a complete response before yielding anything, which can never happen for a genuinely unbounded SSE generator. Confirmed non-viable rather than fought further; the SSE tests instead drive a **real subprocess uvicorn server** over a real TCP socket — closer to what a judge's browser or `curl -N` actually does, and the same mechanism already manually verified by hand.

Live verification performed (beyond the automated tests)

- `curl -N` against `/api/twin/stream`: `run_meta` first, then `snapshot` events at ~6.7 frames/s against the 8 Hz ticker (reasonable given per-frame HTTP/SSE overhead), ticks strictly increasing.
 - Malformed control payloads: unknown station → 404; out-of-range multiplier → 422; missing required field → 422.
 - Three simultaneous browser-equivalent clients (concurrent `curl -N`) ended in exact sync at the same tick and seq.
 - A control command posted against a live server was reflected in `/api/twin/state` well within the 2-second live-response requirement — asserted directly in `test_control_then_state_reflect_the_change_live`, not just eyeballed.
 - Restart: `run_id` changes, tick resets, and — deliberately tested, not merely assumed — a **faulted** engine (a synthetic exception injected into `env.run()`) recovers to `status: "running"` after a restart, rather than staying stuck faulted.
-

Exit criteria

Criterion	Status
<code>sim_time_s == tick * SIM_DT</code> exactly	Met
Real-time factor 1.00 ± 0.05	Met — measured 0.999 in isolated testing
Single-slot conflation bus; a slow/dead client cannot stall the clock	Met — proven by a direct test (<code>test_a_slow_waiter_never_blocks_publish</code>), not just asserted
<code>curl -N</code> verified live; malformed payloads rejected; 3 concurrent clients stay in sync	Met
Control command applies within the next tick and is observably reflected	Met
Restart recovers a running and a faulted engine	Met
Dark stations report honestly (no fabricated inference before Phase 9 exists)	Met
No P0 defect survives to the next phase	Met — the deadlock was found, root-caused, fixed, and regression-tested before this phase closed

96/96 tests passing (70 from Phases 1–5 + 26 new this phase).

Next

Phase 7 — Floor-Supervisor View. The first of three persona views: the real-time dashboard a floor supervisor actually watches, built on this phase's transport layer. Nothing in it counts as done until the perturbation-to-visible-change loop has been watched happening in an actual browser.